

ROLE	TIMEBOX	OUTPUT
Unity Developer (Senior)	One working day, about 8 hours	Repo + build + two 1-page notes

Context

NumTalk is an early-stage mobile product. This assignment is not that product.

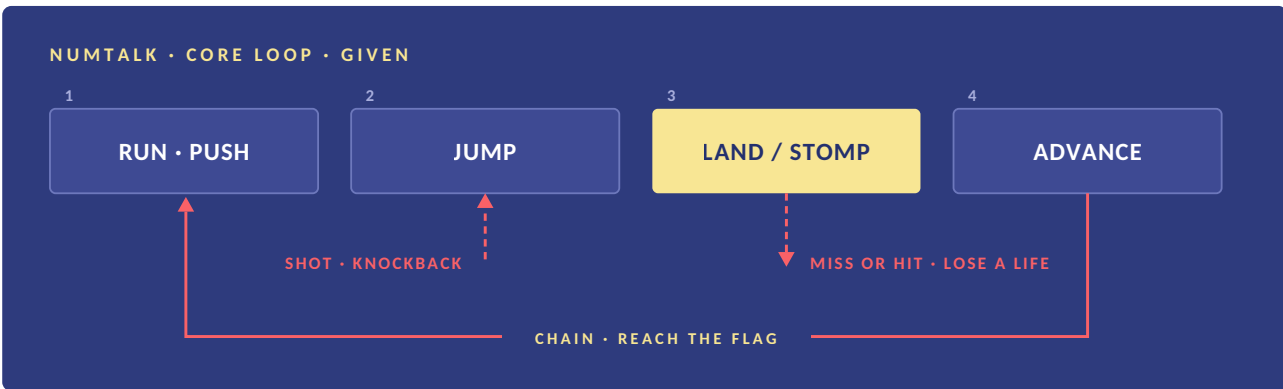
It is a self-contained gameplay prototype, and we use it to see one thing: how you build and tune a character-driven 3D platformer for touch, and what you choose to cut when the clock is real. We are hiring a senior, so we are reading judgement rather than volume.

What this brief asks of you

We are not asking you to design a game. The genre is given, a 3D platformer in the Mario lineage, and the Core Loop below is ours: four beats, in this order.

We are asking you to make it **feel** right: the jump, the thumb, the camera, the moment the platform stops being trustworthy, and **every point at which two physical things meet**. Everything below the loop is a decision we are handing to you on purpose.

The Core Loop, given



1 RUN · PUSH	One thumb steers. Movement is camera-relative , with real acceleration and deceleration, not an instant snap and not a slide that never ends. Walking into something moves it.
2 JUMP	The other thumb commits. Height is variable: release early, hop lower. The player should feel the game was on their side even when they were slightly late.
3 LAND / STOMP	Land on a platform, or land on an enemy's head and bounce. The platform answers back: it moves, it is slick, or it gives way.
4 ADVANCE	The camera settles and the next gap, and whatever is wrong with it, is legible before the player commits again.
SHOT	An enemy projectile does not cost a life. It moves the player . An external force enters the loop and the character has to absorb it without the player losing control.
MISS OR HIT	Fall out of the world, or take an enemy from the side: lose one of three lives, respawn at the last checkpoint. This is the cost that makes the jump worth timing.

The game, in one paragraph

A character runs and jumps along a short course of floating platforms: gaps to clear, patrolling enemies to stomp, coins to pick up, crates to shove out of the way, shooters that knock you off your line, platforms that move, ice over and crumble

underfoot, and a flag at the end. **Built-in primitives only.** A capsule, some cubes and some spheres is exactly the right fidelity for this. We want the *feel* of the genre, not a reproduction of any particular game.

What is yours to choose

The movement model is deliberately not specified. These are the calls we want you to make, and to defend in one page:

- *CharacterController*, *Rigidbody*, or your own kinematic mover, and how you keep the result frame-rate independent whichever you pick.
- How much control the player has in the air, and whether a jump can be cancelled. Both answers are defensible; a vague answer is not.
- How the camera works: pure follow, player-steered by drag, or a hybrid, and what it does when a platform gets between it and the player.
- How the player rides a moving platform, and what velocity they keep when they jump off it.
- **How external forces enter your movement model**, a knockback impulse or the resistance of a crate, without the player losing control of the character, and how they decay.
- Whether a pushable object is a *Rigidbody* you apply force to, a kinematic body you sweep yourself, or something else entirely.
- How "a platform that misbehaves" is modelled, so that the fourth behaviour costs you an hour and not a day.

What to build

MUST: THIS IS THE ASSIGNMENT

1. **Run.** A floating virtual stick: the thumb lands anywhere in the left region and the stick appears there, then drag steers. Camera-relative, with acceleration and deceleration you have actually tuned. Both thumbs must work at the same time.
2. **Jump.** Tap the right region. **Variable height:** release early, hop lower. **Coyote time** and **jump buffering** both implemented, both as named, tunable values. Tell us the numbers you settled on and why.
3. **Camera.** Third-person follow with damping and a little lead. It must not fight the player mid-jump, and it must not leave the player off-screen or behind geometry at the moment they have to commit.
4. **Land and stomp.** Land on an enemy from above and the enemy dies while the player bounces; holding jump gives a higher bounce. Contact from the side or below and the player takes the hit. This discrimination has to hold up at full falling speed *and* at 30 fps.
5. **Platforms that misbehave.** **Moving** (the player rides it and keeps sensible velocity when jumping off), **Ice** (low friction, so momentum carries and can carry you off the edge) and **Crumble** (falls away shortly after it is stood on). Composable behaviours on one platform type, not three forked prefabs. **If your design is right, the third of these costs you fifteen minutes. That is the part we are measuring.**
6. **Fail and checkpoint.** Fall out of the world or take a hit, lose one of three lives, respawn at the last checkpoint with world state restored to something coherent. Say what you restore and what you deliberately do not. At zero lives the run ends and restarts cleanly, including the third restart.
7. **The course.** One authored course, 60 to 120 seconds played well, ending at a goal. Publish your tuned **max jump height** and **max jump distance**, and build every gap so it clears with margin.
8. **Frame-rate independence, and no tunneling.** The same feel at 30, 60 and 120 fps. Falling at terminal velocity must not put the player through a platform. Tell us how you ensured both.
9. **It runs on a phone.** Android APK, TestFlight, or a device screen recording. Two thumbs on glass, not a keyboard.

MUST: COLLISION

These three are the part of the submission we open first.

1. **Push.** At least one landing platform is blocked: a crate sits exactly where the player needs to come down, and walking into it is what moves it. The crate is a real physical object. It has mass and friction, it can be shoved off an edge, it can be pushed onto ice and keep going, and the player can stand on it and jump from it. It must never end up inside a wall or through the floor, and it must never push the player through anything either. If you are using *CharacterController*, note that pushing does not come for free, so tell us what you did.
2. **Get shot.** At least one enemy fires a projectile. A hit does not cost a life. It **moves the player**: a knockback impulse your movement model has to absorb and decay *without the player losing control of the character*. It has to compose with everything already in the game: hit while airborne, hit while standing on ice, hit while riding a moving platform, hit while leaning on the crate. Being knocked off a ledge is a legitimate death.
3. **Collision hygiene.** Projectiles must not tunnel through platforms or through the player at full speed, must not hit the enemy that fired them, and must not hit each other. Do that with the **layer collision matrix** and the right collision detection mode, not with tag comparisons inside *OnTriggerEnter*. Put your layer layout in the README.

SHOULD: IF THE CORE IS SOLID

1. **One sudden event that changes the rules mid-run.** A section that flash-freezes, a run of platforms that begins crumbling ahead of you, a shooter that wakes up as you land. Telegraphed, readable and fair: the player has to be able to *respond*, not merely be punished.
2. **One tuning asset.** A single *ScriptableObject* holding move speed, acceleration, jump height, gravity up versus down, coyote time, buffer window, ice friction, crumble delay, crate mass, knockback impulse and decay, so we can retune the game without opening a script.
3. **Feel.** Squash and stretch, a landing that reads, a coin pop, a stomp bounce and a knockback you can see coming.

BONUS: ONLY IF THE ABOVE IS GENUINELY DONE

1. **Camera occlusion.** The camera pulls in, or the blocking geometry fades, when something gets between it and the player.
2. **One move of your own.** Wall jump, ground pound, dash, double jump. Pick one and make it belong.
3. **Deterministic replay.** Recorded input reproduces a run exactly. Prove it with a "replay last run" button.
4. **Zero per-frame GC allocation** in the movement loop, with projectiles, enemies and pickups pooled. Show us the Profiler screenshot.

Stop at eight hours. If you have to cut, cut in this order: bonus, then feel, then the sudden event, then course length. **Do not cut the collision block.** Ship what works and write down what you would have done next, which is a better answer than a half-landed feature.

Out of scope

Menus beyond a start and a game-over overlay · art, models, textures, shaders · audio · saves, leaderboards, analytics, ads · localisation · a settings screen · more than one course · test coverage beyond what actually helped you build it.

Do not spend the day on UI. A coin counter and three life pips drawn with default components is enough. Time spent on polish there is time taken from the part we are reading.

Constraints

- Unity 6.x, 6.3 LTS or newer preferred, any recent LTS accepted. State the version in the README.
- Built-in primitives and free assets only. No package, template or asset that ships a third-person platformer controller for you.
- **No Nintendo assets, characters, names, music or level layouts.** The genre is the reference; the content is yours.
- Input System package preferred over the legacy *Input* class. If you go the other way, say why.
- One playable scene. A git repository with a readable commit history, not a single "initial commit".

Deliverables

1. **Repository.** Link or zip.
2. **Build.** APK, TestFlight link, or a screen recording under 90 seconds taken on a real device.
3. **README,** one page. Unity version, how to run it, what is in and what is out, known issues, your tuned movement numbers, and your layer layout.
4. **DECISIONS.md,** one page. The three hardest calls you made and the trade-off in each; where you would take the controller next; and the AI note below.

On AI use

Use whatever assistant you use on a normal working day. We are not testing whether you can type, and a one-day timebox assumes you are not working unaided. In *DECISIONS.md*, tell us three things:

- Roughly what share of the code you generated, versus wrote or substantially rewrote yourself.
- One thing your assistant got wrong **in this task**, and how you caught it.
- One suggestion you rejected, and why.

We will go through this code with you afterwards. A candidate who ships generated code they cannot walk through line by line will not get past that conversation.

We already know how to make a capsule jump onto a cube. What we want to see is the minute after: how the jump sits in the thumb, what happens the first time something shoves the character while the player is still holding the stick, and whether the fourth platform behaviour costs you an hour or a day.